



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO®



Instituto Tecnológico de San Juan del Río



Tópicos de ciberseguridad

Hardening servidor Ubuntu

P R E S E N T A:

ABELARDO GARDUÑO FUERTES 22590040

ALEJANDRO PÉREZ PIÑA 22590068

BRANDON ISMAEL TREJO HERNÁNDEZ 22590049

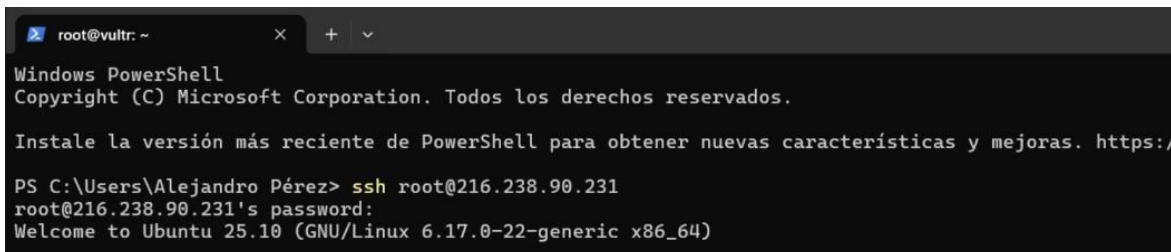
PERIODO [ENE – JUN 2026]

ÍNDICE

1. BLOQUEO PUERTO 22 SSH.....	3
2. CREAR USUARIO CON PERMISOS DE ADMINISTRADOR	8
3. BLOQUEO ACCESO CON ROOT.....	9
4. CLAVE PÚBLICA Y PRIVADA	11
5. GOOGLE AUTHENTICATOR	12
6. RESPALDO EN LA CARPETA DE LA API	19
7. RESPALDO FUERA DE LA CARPETA DE LA API.....	22
8. RESPALDO HACIA LA NUBE (GOOGLE DRIVE).....	23

1. BLOQUEO PUERTO 22 SSH

Paso 1. Accedemos al servidor mediante una línea de comandos (por ejemplo, CMD). Cabe destacar que en este paso se puede acceder por SSH en el puerto 22 (El comando de acceso se muestra en la figura 1).



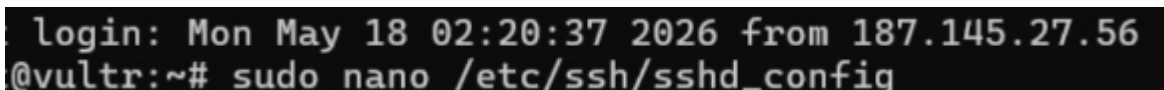
```
root@vultr: ~
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://

PS C:\Users\Alejandro Pérez> ssh root@216.238.90.231
root@216.238.90.231's password:
Welcome to Ubuntu 25.10 (GNU/Linux 6.17.0-22-generic x86_64)
```

Figura 1. Acceso al servidor.

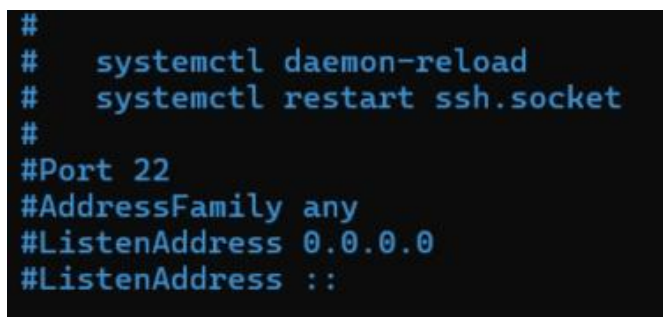
Paso 2. Accedemos al archivo de configuración de SSH mediante el uso de nano (ver figura 2).



```
login: Mon May 18 02:20:37 2026 from 187.145.27.56
@vultr:~# sudo nano /etc/ssh/sshd_config
```

Figura 2. Comando acceso archivo de configuración.

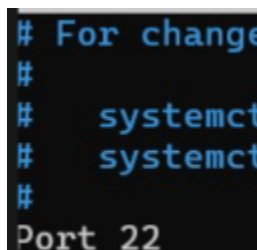
Paso 3. Nos desplazamos con la flechita que apunta hacia abajo en nuestro teclado, y ubicamos la línea que diga “Port 22” (ver figura 3).



```
#
#   systemctl daemon-reload
#   systemctl restart ssh.socket
#
#Port 22
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::
```

Figura 3. Línea Port 22.

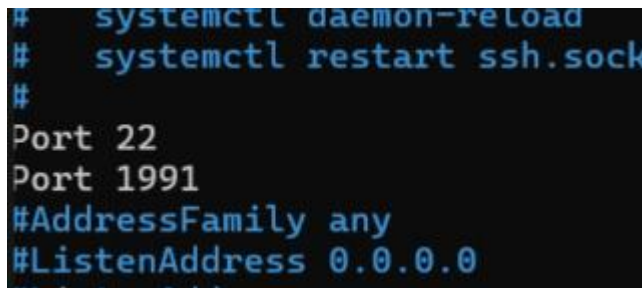
Paso 4. Borramos el “#” que se encuentra al lado de la línea “Port 22” (ver figura 4).



```
# For change
#
# systemct
# systemct
#
Port 22
```

Figura 4. Eliminación del #.

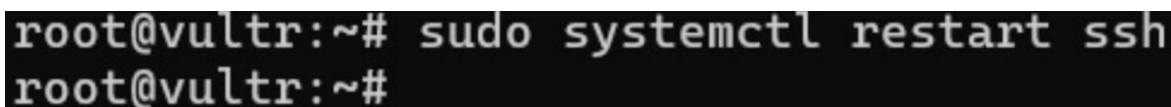
Paso 5. Agregamos el nuevo puerto una línea debajo escribiendo “Port y el número del puerto que se desea usar”, por ejemplo, Port 1991 (ver figura 5). Para guardar el archivo apretamos las siguientes teclas: ctrl + O, enter, ctrl + X.



```
# systemctl daemon-reload
# systemctl restart ssh.sock
#
Port 22
Port 1991
#AddressFamily any
#ListenAddress 0.0.0.0
```

Figura 5. Agregación del nuevo puerto.

Paso 6. Reiniciamos el servicio SSH (el comando se encuentra en la figura 6).



```
root@vultr:~# sudo systemctl restart ssh
root@vultr:~#
```

Figura 6. Reinicio del servicio SSH.

Paso 7. Verificamos el status de las reglas del firewall (ver figura 7).

```
root@vultr:~# sudo ufw status
Status: active

To Action From
--
22/tcp ALLOW Anywhere
8080/tcp ALLOW Anywhere
22/tcp (v6) ALLOW Anywhere (v6)
8080/tcp (v6) ALLOW Anywhere (v6)
```

Figura 7. Estatus del firewall.

Paso 8. Activamos la regla de firewall para permitir el acceso por el puerto 1991 (ver figura 8).

```
root@vultr:~# sudo ufw allow 1991/tcp comment 'SSH Chun-Li'
Rule added
Rule added (v6)
root@vultr:~#
```

Figura 8. Activación regla del firewall.

Paso 9. Verificamos si la regla se ha agregado usando el comando del paso 7 (ver figura 9).

```
root@vultr:~# sudo ufw status
Status: active

To Action From
--
22/tcp ALLOW Anywhere
8080/tcp ALLOW Anywhere
1991/tcp ALLOW Anywhere # SSH Chun-Li
22/tcp (v6) ALLOW Anywhere (v6)
8080/tcp (v6) ALLOW Anywhere (v6)
1991/tcp (v6) ALLOW Anywhere (v6) # SSH Chun-Li
root@vultr:~#
```

Figura 9. Agregación regla del firewall.

Paso 10. Reiniciamos el firewall para activar los cambios (ver figura 10).

```
root@vultr:~# sudo ufw reload
Firewall reloaded
root@vultr:~#
```

Figura 10. Agregación regla del firewall.

Paso 11. Volvemos a acceder al archivo de configuración de SSH mediante el uso de nano como en el paso 2, nos desplazamos con la flechita que apunta hacia abajo en nuestro teclado, y ubicamos nuevamente la línea “Port 22”, y antes de esa línea agregamos un “#” (ver figura 11).

```
#
#Port 22
Port 1991
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::
```

Figura 11. Agregar # a Port 22.

Paso 12. Reiniciamos el servicio SSH como en el paso 6.

Paso 13. Negamos el acceso al puerto 22 desde las reglas del firewall (ver figura 12).

```
root@vultr:~# sudo ufw deny 22/tcp
Rule updated
Rule updated (v6)
root@vultr:~#
```

Figura 12. Denegar puerto 22.

Paso 14. Reiniciamos las reglas del firewall como en el paso 10.

Paso 15. Checamos el estado final de las reglas del firewall (ver figura 13).

```
root@vultr:~# sudo ufw status
Status: active

To Action From
--
22/tcp DENY Anywhere
8080/tcp ALLOW Anywhere
1991/tcp ALLOW Anywhere # SSH Chun-l
443/tcp ALLOW Anywhere
22/tcp (v6) DENY Anywhere (v6)
8080/tcp (v6) ALLOW Anywhere (v6)
1991/tcp (v6) ALLOW Anywhere (v6) # SSH Chun-l
443/tcp (v6) ALLOW Anywhere (v6)
```

Figura 13. Checar regla del firewall.

Demostración: Si intentamos acceder nuevamente al servidor por el puerto 22, nos denegará el acceso (ver figura 14).

```
instale la version mas reciente de powershell para obtener nuevas c
PS C:\Users\Alejandro Pérez> ssh root@216.238.90.231
ssh: connect to host 216.238.90.231 port 22: Connection timed out
```

Figura 14. Denegación acceso puerto 22.

En cambio, si accedemos por el puerto que agregamos (1991), nos permitirá el acceso (ver figura 15).

```
PS C:\Users\Alejandro Pérez> ssh -p 1991 root@216.238.90.231
root@216.238.90.231's password:
Permission denied, please try again.
root@216.238.90.231's password:
Welcome to Ubuntu 25.10 (GNU/Linux 6.17.0-22-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Fri May 22 00:18:54 UTC 2026

System load:  0.1          Processes:      160
Usage of /:   37.3% of 22.88GB Users logged in: 0
Memory usage: 32%          IPv4 address for enp1s0: 216.238.90.231
Swap usage:   21%

* Due to a now-resolved bug in the date command, this system may be unable
to automatically check for updates. Manually install the update using:

    sudo apt install --update rust-coreutils

https://discourse.ubuntu.com/t/enabling-updates-on-ubuntu-25-10-systems/
```

Figura 15. Acceso puerto 1991.

2. CREAR USUARIO CON PERMISOS DE ADMINISTRADOR

Paso 1. Creamos el nuevo usuario agregando un nombre de usuario, una contraseña y para las demás preguntas, presionamos la tecla enter para dejar los valores por default (ver figura 16).

```
root@vultr:~# sudo adduser ChunLi
New password:
BAD PASSWORD: The password fails the dictionary check - it is based on a dictionary word
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for ChunLi
Enter the new value, or press ENTER for the default
    Full Name []:
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] Y
root@vultr:~#
```

Figura 16. Creación del nuevo usuario.

Paso 2. Asignamos los privilegios de super usuario al usuario que creamos anteriormente (ver figura 17).

```
root@vultr:~# sudo usermod -aG sudo chunli
root@vultr:~#
```

Figura 17. Asignación de privilegios.

Paso 3. Accedemos al servidor con el nuevo usuario creado (ver figura 18).

```
Usage of /: 37.3% of 22.88GB  Users logged in: 0
Memory usage: 33%          IPv4 address for enp1s0: 216.238.90.231
Swap usage: 21%

* Due to a now-resolved bug in the date command, this system may be unable
to automatically check for updates. Manually install the update using:

    sudo apt install --update rust-coreutils

https://discourse.ubuntu.com/t/enabling-updates-on-ubuntu-25-10-systems/

11 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

New release '26.04 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

*** System restart required ***
*****PLEASE NOTE*****
* If this is a new instance, cloud-init may still be running and normal
* operations may not work as expected.
*
* Please use 'cloud-init status' to check on the status of the instance
* provisioning
*
* Remove this message by editing /etc/motd
*****
Last login: Thu May 21 02:49:00 2026 from 187.145.27.56
chunli@vultr:~$
```

Figura 18. Acceso con el nuevo usuario.

Paso 4. Ejecutamos un “sudo apt update” para verificar que podamos realizar comandos de administrador (ver figura 19).

```
ChunLi@vultr:~$ sudo apt update
[sudo: authenticate] Password:
Hit:1 http://ubuntu.mirror.constant.com questing InRelease
Hit:2 http://security.ubuntu.com/ubuntu questing-security InRelease
Hit:3 http://ubuntu.mirror.constant.com questing-updates InRelease
Hit:4 http://ubuntu.mirror.constant.com questing-backports InRelease
```

Figura 19. Comprobación.

3. BLOQUEO ACCESO CON ROOT

Paso 1. Al acceder con el nuevo usuario, accedemos al archivo de configuración del SSH mediante el uso de nano (ver figura 20).

```
ChunLi@vultr:~$ sudo nano /etc/ssh/sshd_config
```

Figura 20. Acceso con el nuevo usuario a los archivos de configuración ssh.

Paso 2. Nos desplazamos con la flechita que apunta hacia abajo en nuestro teclado, hasta encontrar la línea “PermitRootLogin” y cambiamos el “PermitRootLogin Yes” por “PermitRootLogin No” (ver figura 21). Para guardar el archivo apretamos las siguientes teclas: ctrl + O, enter, ctrl + X.

```
PermitRootLogin no

^G Help      ^O Write Out  ^F Where Is
^X Exit      ^R Read File  ^\ Replace
```

Figura 21. PermitRootLogin No.

Paso 3. Reiniciamos el servicio de SSH (ver figura 22).

```
ChunLi@vultr:~$ sudo systemctl restart ssh
ChunLi@vultr:~$
```

Figura 22. Reinicio del servicio SSH.

Paso 4. Deshabilitamos la contraseña del root para bloquear el acceso al root (ver figura 23).

```
ChunLi@vultr:~$ sudo passwd -l root
passwd: password changed.
ChunLi@vultr:~$
```

Figura 23. Reinicio del servicio SSH.

Demostración: Si intentamos acceder mediante el root al servidor, nos denegará el acceso (ver figura 24).

```
PS C:\Users\Alejandro Pérez> ssh -p 1991 root@216.238.90.231
(root@216.238.90.231) Password:
(root@216.238.90.231) Password:
(root@216.238.90.231) Password:
root@216.238.90.231's password:
Permission denied, please try again.
root@216.238.90.231's password:
Permission denied, please try again.
root@216.238.90.231's password:
```

Figura 24. Acceso denegado al root.

4. CLAVE PÚBLICA Y PRIVADA

Paso 1. En nuestra máquina local, generamos las llaves seguras para generar la clave pública y privada a través de una línea de comandos para evitar el acceso por contraseña (por ejemplo, CMD) (ver figura 25).

```
PS C:\Users\Alejandro Pérez> ssh-keygen -t ed25519 -C "ChunLi-ssh"
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\Alejandro Pérez/.ssh/id_ed25519):
```

Figura 25. Generación de las claves en la máquina local.

Paso 2. Generamos las llaves seguras en nuestro servidor Ubuntu desde nuestra máquina local (ver figura 26).

```
PS C:\Users\Alejandro Pérez> ssh-keygen -t ed25519 -C "ChunLi-ssh"
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\Alejandro Pérez/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\Alejandro Pérez/.ssh/id_ed25519
Your public key has been saved in C:\Users\Alejandro Pérez/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:1XB2hUWRh5Fo1CpgFwY0FB3JNUaZLP70jqRi9Z3LcY ChunLi-ssh
The key's randomart image is:
+--[ED25519 256]--+
|      .++oBX*+ooX*|
|      o*=*o.=..|
|      ..+....  |
|      .o o      |
|      S  + .    |
|      .  o .    |
|      . + o + .  |
|      o + o E .  |
|      ... : o .  |
+-----+
Fingerprint 1XB2hUWRh5Fo1CpgFwY0FB3JNUaZLP70jqRi9Z3LcY
```

Figura 26. Generación de las claves en servidor Windows.

Paso 3. Copiamos la llave generada de manera local a nuestro servidor Ubuntu desde nuestra máquina local (ver figura 27).

```
PS C:\Users\Alejandro Pérez> cat C:\Users\Alejandro Pérez/.ssh/id_ed25519.pub | ssh -p 1991 ChunLi@216.238.90.231 "mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys"
(ChunLi@216.238.90.231) Password:
```

Figura 27. Copiar la llave.

Paso 4. Intentamos acceder nuevamente a nuestro servidor Ubuntu para comprobar que no se acceda por contraseña (ver figura 28).

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras.

PS C:\Users\Alejandro Pérez> ssh -p 1991 ChunLi@216.238.90.231
Welcome to Ubuntu 25.10 (GNU/Linux 6.17.0-22-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri May 22 01:33:57 UTC 2026

System load:  0.0               Processes:            165
Usage of /:   37.4% of 22.88GB   Users logged in:     0
Memory usage: 33%              IPv4 address for enp1s0: 216.238.90.231
Swap usage:   21%

 * Due to a now-resolved bug in the date command, this system may be unable
   to automatically check for updates. Manually install the update using:

       sudo apt install --update rust-coreutils

https://discourse.ubuntu.com/t/enabling-updates-on-ubuntu-25-10-systems/

14 updates can be applied immediately.
4 of these updates are standard security updates.
```

Figura 28. Acceso sin contraseña.

5. GOOGLE AUTHENTICATOR

Paso 1. Descargamos e instalamos el paquete oficial que permite a Ubuntu entender los códigos de Google (ver el código en la figura 29).

```
Last login: Fri May 22 01:34:10 2026 from 30.43.133.221
ChunLi@vultr:~$ sudo apt update && sudo apt install libpam-google-authenticator -y
[sudo: authenticate] Password:
Hit:1 http://ubuntu.mirror.constant.com questing InRelease
Get:2 http://ubuntu.mirror.constant.com questing-updates InRelease [136 kB]
Hit:3 http://security.ubuntu.com/ubuntu questing-security InRelease
Hit:4 http://ubuntu.mirror.constant.com questing-backports InRelease
Get:5 http://ubuntu.mirror.constant.com questing-updates/main amd64 Packages [437 kB]
Get:6 http://ubuntu.mirror.constant.com questing-updates/universe amd64 Packages [323 kB]
Fetched 895 kB in 1s (919 kB/s)
11 packages can be upgraded. Run 'apt list --upgradable' to see them.
Installing:
  libpam-google-authenticator

Installing dependencies:
  libqrencode4

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 11
```

Figura 29. Paquete oficial de Google.

Paso 2. Una vez descargado, generamos la clave secreta para el usuario creado, el cual, ejecutara el asistente de Google authenticator (ver figura 30).

```
ChunLi@vultr:~$ google-authenticator
```

Figura 30. Ejecutar asistente de Google authenticator.

Paso 3. Se generará una serie de preguntas, las cuales se tienen que responder presionando la tecla correspondiente (Y para si / N para no) y después presionando la tecla enter. Estas preguntas son:

- Pregunta 1: Do you want authentication tokens to be time-based (y/n). Presionamos “y” y damos enter (ver figura 31).

```
Do you want authentication tokens to be time-based (y/n) y
```

Figura 31. Primera pregunta.

Se generará un código QR. Ese código se tiene que escanear desde la aplicación, para ello, desde la tienda de tu teléfono se descarga la aplicación de Google authenticator, y en ella, se le da al símbolo de “+” en la parte inferior izquierda, y en “Escanear código QR” (ver figura 32).

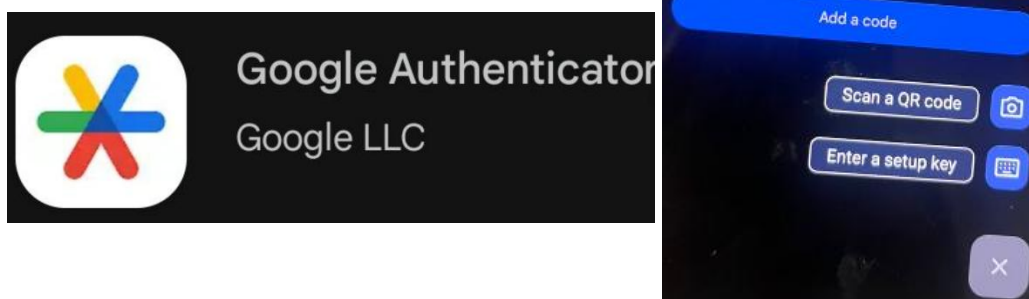


Figura 32. Aplicación y opción.

Al escanear el código QR, se generarán 5 códigos de emergencia en dado caso que se pierda el acceso a la cuenta de Google (figura 33).

```
Code confirmed
Your emergency scratch codes are:
29834953
29430629
26360802
67495829
58375565
```

Figura 33. Códigos de emergencia.

- Pregunta 2: Do you want me to update your "/home/ChunLi/.google_authenticator" file? (y/n). Presionamos "y" y damos enter (ver figura 34).

```
Do you want me to update your "/home/ChunLi/.google_authenticator" file? (y/n) y
```

Figura 34. Segunda pregunta.

- Pregunta 3: Do you want to disallow multiple uses of the same authentication token? (y/n). Presionamos "y" y damos enter (ver figura 35).

```
Do you want to disallow multiple uses of the same authentication
token? This restricts you to one login about every 30s, but it increases
your chances to notice or even prevent man-in-the-middle attacks (y/n) y

By default, a new token is generated every 30 seconds by the mobile app.
In order to compensate for possible time-skew between the client and the server,
we allow an extra token before and after the current time. This allows for a
time skew of up to 30 seconds between authentication server and client. If you
experience problems with poor time synchronization, you can increase the window
from its default size of 3 permitted codes (one previous code, the current
code, the next code) to 17 permitted codes (the 8 previous codes, the current
code, and the 8 next codes). This will permit for a time skew of up to 4 minutes
between client and server.
```

Figura 35. Tercera pregunta.

- Pregunta 4: By default, a new token is valid for 30 seconds... Do you want to do so? (y/n). Presionamos “n” y damos enter (ver figura 36).

```
Do you want to do so? (y/n) n

If the computer that you are logging into isn't hardened against brute-force
login attempts, you can enable rate-limiting for the authentication module.
By default, this limits attackers to no more than 3 login attempts every 30s.
```

Figura 36. Cuarta pregunta.

- Pregunta 5: Do you want to enable rate-limiting? (y/n). Presionamos “y” y damos enter (ver figura 37).

```
Do you want to enable rate-limiting? (y/n) y
ChunLi@vultr:~$
```

Figura 37. Quinta pregunta.

Paso 4. Una vez creado el token, se tiene que configurar el servidor a pedirlo, para ello, editamos el archivo de configuración PAM con nano a través del comando: `sudo nano /etc/pam.d/sshd` (ver figura 38).

```
GNU nano 8.4 /etc/pam.d/sshd
# PAM configuration for the Secure Shell service

# Standard Unix authentication.
@include common-auth

# Disallow non-root logins when /etc/nologin exists.
account required pam_nologin.so

# Uncomment and edit /etc/security/access.conf if you need to set complex
# access limits that are hard to express in sshd_config.
# account required pam_access.so

# Standard Unix authorization.
@include common-account

# SELinux needs to be the first session rule. This ensures that any
# lingering context has been cleared. Without this it is possible that a
# module could execute code in the wrong domain.
session [success=ok ignore=ignore module_unknown=ignore default=bad] pam_selinux.so close

# Set the loginuid process attribute.
session required pam_loginuid.so

# Create a new session keyring.
session optional pam_keyinit.so force revoke

# Standard Unix session setup and teardown.
@include common-session

# Print the message of the day upon successful login.
# This includes a dynamically generated part from /run/motd.dynamic
# and a static (admin-editable) part from /etc/motd.
session optional pam_motd.so motd=/run/motd.dynamic
session optional pam_motd.so noupdate

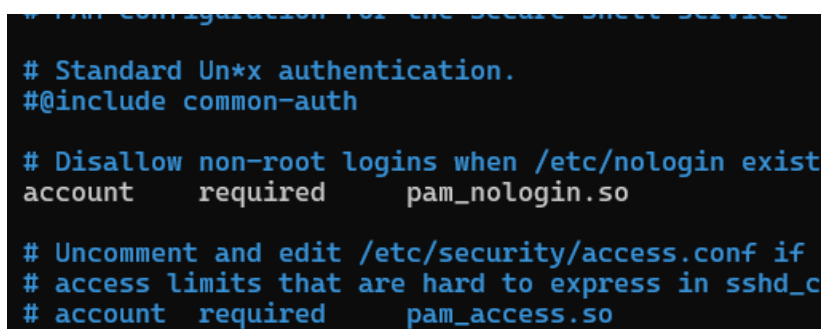
# Print the status of the user's mailbox upon successful login.
session optional pam_mail.so standard noenv # [1]

# Set up user limits from /etc/security/limits.conf.
session required pam_limits.so

# Read environment variables from /etc/environment and
# /etc/security/pam_env.conf.
session required pam_env.so # [1]
# In Debian 4.0 (etch), locale-related environment variables were moved to
# /etc/default/locale, so read that as well.
```

Figura 38. Archivo de configuración.

Paso 5. Con la flechita que apunta hacia abajo en nuestro teclado, buscamos la línea que dice “@include common-auth”. Nos colocamos al inicio de la línea y escribimos el símbolo “#” (ver figura 39). Esto es importante para que el servidor no se confunda pidiendo contraseñas antes de tiempo.



```
# PAM configuration for the Secure Shell Service

# Standard Un*x authentication.
#@include common-auth

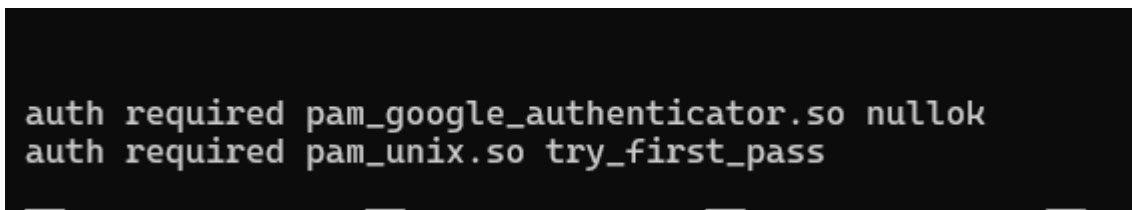
# Disallow non-root logins when /etc/nologin exists
account      required      pam_nologin.so

# Uncomment and edit /etc/security/access.conf if
# access limits that are hard to express in sshd_config
# account      required      pam_access.so
```

Figura 39. Línea editada.

Paso 6. Seguimos desplazándonos con la flechita hacia debajo de nuestro teclado hasta el final del archivo y escribimos las siguientes líneas: (ver figura 40)

- auth required pam_google_authenticator.so nullok
- auth required pam_unix.so try_first_pass

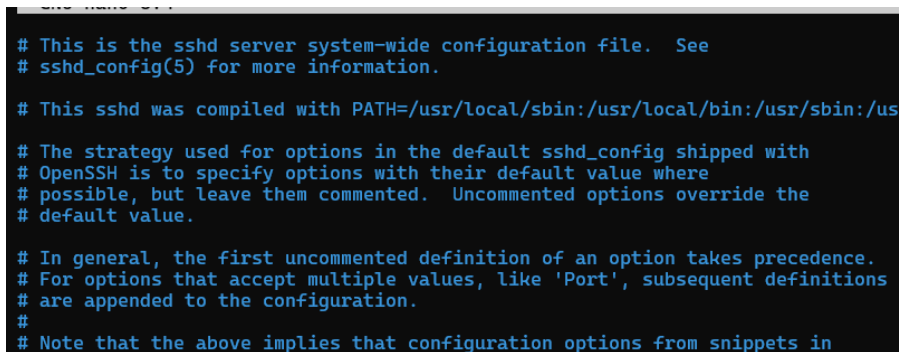


```
auth required pam_google_authenticator.so nullok
auth required pam_unix.so try_first_pass
```

Figura 40. Líneas agregadas.

Guardamos presionando las teclas “Ctrl + O”, luego “Enter” para confirmar, y “Ctrl + X”.

Paso 7. Le indicamos al SSH las reglas de entrada. Para ello, abrimos el archivo de configuración SSH a través de nano con el comando: “sudo nano /etc/ssh/sshd_config” (ver figura 41).



```
# This is the sshd server system-wide configuration file. See
# sshd_config(5) for more information.

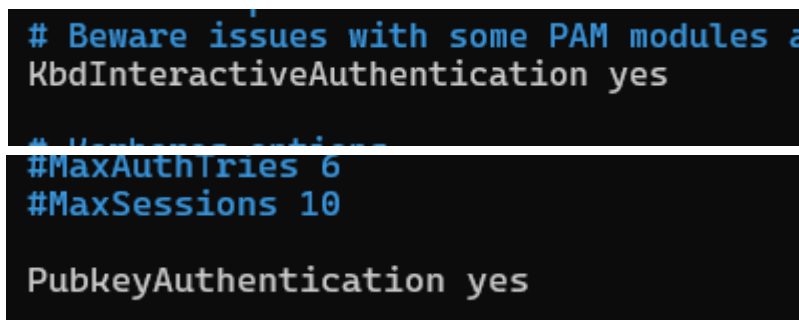
# This sshd was compiled with PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/us

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented. Uncommented options override the
# default value.

# In general, the first uncommented definition of an option takes precedence.
# For options that accept multiple values, like 'Port', subsequent definitions
# are appended to the configuration.
#
# Note that the above implies that configuration options from snippets in
```

Figura 41. Archivo de configuración.

Paso 8. Nos desplazamos con la flechita que apunta hacia debajo de nuestro teclado para asegurarnos que las siguientes líneas estén en “yes”. Si están comentadas con un “#”, quitamos el símbolo (ver figura 42).



```
# Beware issues with some PAM modules a
KbdInteractiveAuthentication yes

# MaxAuthTries 6
# MaxSessions 10

PubkeyAuthentication yes
```

Figura 42. Líneas a verificar.

Paso 9. Seguimos desplazándonos con la flechita hacia debajo de nuestro teclado hasta el final del archivo y escribimos la siguiente línea: “AuthenticationMethods publickey,keyboard-interactive” (ver figura 43). Guardamos presionando las teclas “Ctrl + O”, luego “Enter” para confirmar, y “Ctrl + X”.

```
AuthenticationMethods publickey,keyboard-interactive
```

Figura 43. Línea a agregar.

Paso 10. Reiniciamos el servicio de SSH (ver figura 44).

```
ChunLi@vultr:~$ sudo systemctl restart ssh
ChunLi@vultr:~$ |
```

Figura 44. Reinicio del servicio.

Demostración: Ingresamos por SSH a nuestro servidor Ubuntu, una vez ingresado, nos pedirá primero el código de verificación, el cual, son 6 números que nos genera la app de Google authenticator (ver figura 45).

```
PS C:\Users\Alejandro Pérez> ssh -p 1991 ChunLi@216.238.90.231
(ChunLi@216.238.90.231) Verification code: |
```

Figura 45. Código de verificación.

Después de ingresar el código de verificación, pedirá nuestra contraseña y si todo esta correcto, podremos acceder (ver figura 46).

```
PS C:\Users\Alejandro Pérez> ssh -p 1991 ChunLi@216.238.90.231
(ChunLi@216.238.90.231) Verification code:
(ChunLi@216.238.90.231) Password:
Welcome to Ubuntu 25.10 (GNU/Linux 6.17.0-22-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Sun May 24 09:29:54 UTC 2026

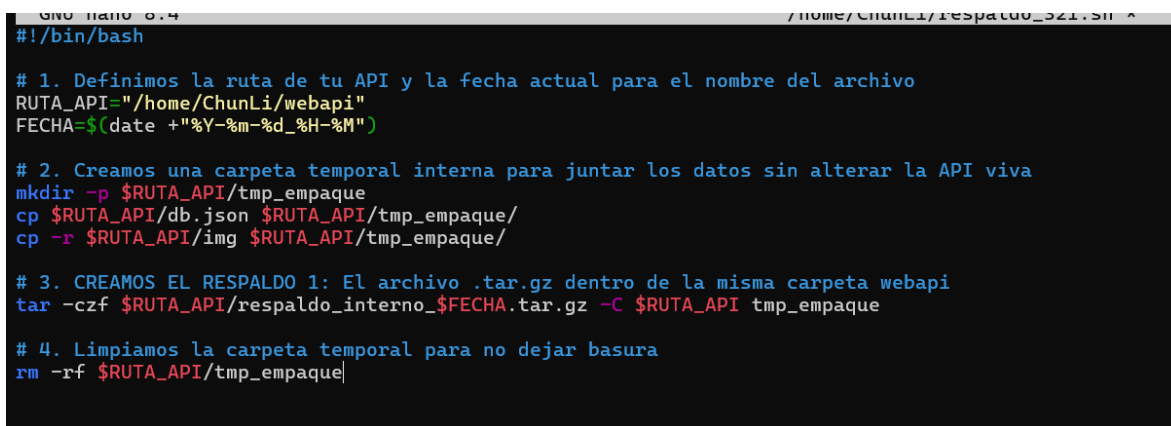
System load:  0.01               Processes:    157
Usage of /:   27.5% of 22.8GB     Memory used: 0
```

Figura 46. Ingreso al servidor.

6. RESPALDO EN LA CARPETA DE LA API

Paso 1. En la carpeta donde guardamos nuestra API, creamos un archivo con ayuda de nano para ingresar el código de nuestro respaldo, por ejemplo: “nano /home/ChunLi/respaldo_321.sh”

Paso 2. Ingresamos las líneas que se observan en la figura 47, las cuales, se encargan de juntar nuestros archivos y meterlos en un paquete comprimido dentro de la misma carpeta. Guardamos presionando las teclas “Ctrl + O”, luego “Enter” para confirmar, y “Ctrl + X”.



```
GNU nano 2.9.4 /home/ChunLi/respaldo_321.sh
#!/bin/bash

# 1. Definimos la ruta de tu API y la fecha actual para el nombre del archivo
RUTA_API="/home/ChunLi/webapi"
FECHA=$(date +"%Y-%m-%d_%H-%M")

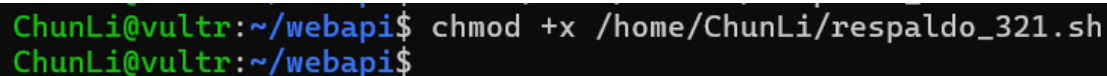
# 2. Creamos una carpeta temporal interna para juntar los datos sin alterar la API viva
mkdir -p $RUTA_API/tmp_empaque
cp $RUTA_API/db.json $RUTA_API/tmp_empaque/
cp -r $RUTA_API/img $RUTA_API/tmp_empaque/

# 3. CREAMOS EL RESPALDO 1: El archivo .tar.gz dentro de la misma carpeta webapi
tar -czf $RUTA_API/respaldo_interno_$FECHA.tar.gz -C $RUTA_API tmp_empaque

# 4. Limpiamos la carpeta temporal para no dejar basura
rm -rf $RUTA_API/tmp_empaque
```

Figura 47. Líneas a ingresar.

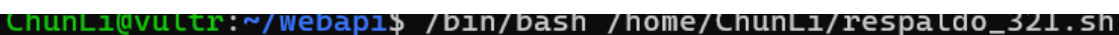
Paso 3. Le damos permisos a ese archivo para poder correr el script como programa (ver figura 48). El “+x” le da permisos de ejecución al archivo.



```
ChunLi@vultr:~/webapi$ chmod +x /home/ChunLi/respaldo_321.sh
ChunLi@vultr:~/webapi$
```

Figura 48. Otorgar permisos.

Demostración: corremos el script con el comando de la figura 49.



```
ChunLi@vultr:~/webapi$ /bin/bash /home/ChunLi/respaldo_321.sh
```

Figura 49. Comando a ejecutar.

Una vez ejecutado, ejecutamos un ls para comprobar que el archivo se generó correctamente (ver figura 50).

```
ChunLi@vultr:~/webapi$ ls -lh /home/ChunLi/webapi/
total 172K
-rw-r--r-- 1 ChunLi ChunLi 1.7K May  1 00:50 216.238.90.231-key.pem
-rw-r--r-- 1 ChunLi ChunLi 1.6K May  1 00:50 216.238.90.231.pem
-rw-r--r-- 1 ChunLi ChunLi 101K May 24 09:46 api.log
-rw-r--r-- 1 ChunLi ChunLi 1.9K May  1 00:10 cert.pem
-rw-r--r-- 1 ChunLi ChunLi 7.6K May  5 19:58 cpindex.py
-rw-r--r-- 1 ChunLi ChunLi  88 Apr 27 11:16 db.json
drwxr-xr-x 2 ChunLi ChunLi 4.0K Apr 30 23:51 img
-rw-r--r-- 1 ChunLi ChunLi 9.4K May  5 19:59 index.py
-rw-r--r-- 1 ChunLi ChunLi 3.2K May  1 00:09 key.pem
-rw-rw-r-- 1 ChunLi ChunLi 11K May 24 09:55 respaldo_interno_2026-05-24_09-55.tar.gz
drwxr-xr-x 2 ChunLi ChunLi 4.0K Apr 30 23:51 tmp
drwxr-xr-x 5 ChunLi ChunLi 4.0K Apr 27 11:20 venv
ChunLi@vultr:~/webapi$
```

Figura 50. Comprobación del respaldo.

Paso 4. Para programar el respaldo a la 1:01 a.m., ejecutamos el comando de la figura 51. Si es la primera vez que se abre, preguntará qué editor usar. Presionamos el número que corresponda a nano (usualmente el 1) y pulsamos la tecla Enter.

```
ChunLi@vultr:~$ nano /home/ChunLi/respaldo_521.sh
ChunLi@vultr:~$ crontab -e
no crontab for ChunLi - using an empty one
Select an editor. To change later, run select-editor again.
 1. /bin/nano      <---- easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny
 4. /bin/ed

Choose 1-4 [1]:
```

Figura 51. Comando a ejecutar.

Paso 5. Con la flechita que apunta hacia debajo de nuestro teclado, bajamos hasta el final del archivo y pegamos la línea que se muestra en la figura 52.

```
GNU nano 0.4 /tmp/crontab.wE7xUj/crontab
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tar.gz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
1 1 * * * /bin/bash /home/ChunLi/respaldo_321.sh > /home/ChunLi/respaldo_cron.log 2>&1
```

Figura 52. Línea a agregar.

El significado de la línea es la siguiente:

- 1 1 * * * → Significa: Minuto 1, Hora 1 (es decir, 1:01 AM), todos los días, todos los meses.
- /bin/bash ... → Arranca el script de respaldos automáticamente.
- > /home/ChunLi/... → Guarda un reporte silencioso por si algún día se quiere revisar que se ejecutó bien.

Por último, Guardamos presionando las teclas “Ctrl + O”, luego “Enter” para confirmar, y “Ctrl + X”.

Demostración: ejecutamos un ls para comprobar que el archivo se generó correctamente a la hora establecida (ver figura 53).

```
ChunLi@vultr:~/webapi$ ls
216.238.90.231-key.pem  img                                respaldo_interno_2026-05-25_01-01.tar.gz
216.238.90.231.pem      index.py                          respaldo_interno_2026-05-26_01-01.tar.gz
api.log                 key.pem                           respaldo_interno_2026-05-26_11-27.tar.gz
cert.pem                respaldo_interno_2026-05-24_09-55.tar.gz  respaldo_interno_2026-05-26_11-28.tar.gz
cpindex.py              respaldo_interno_2026-05-24_10-27.tar.gz  tmp
db.json                 respaldo_interno_2026-05-24_10-57.tar.gz  venv
ChunLi@vultr:~/webapi$
```

Figura 53. Respaldo 1 a.m.

7. RESPALDO FUERA DE LA CARPETA DE LA API

Paso 1. Creamos la carpeta externa donde deseamos guardar el respaldo a través del comando: “mkdir -p /home/ChunLi/respaldos_locales”

Paso 2. En el archivo que creamos en la sección 6, paso 2 de este documento, lo modificamos para que genere también el paquete en la nueva ruta y lo comprima a “.tar” (ver figura 54). Guardamos presionando las teclas “Ctrl + O”, luego “Enter” para confirmar, y “Ctrl + X”.

```
#!/bin/bash

# 1. Definimos las rutas y la fecha
RUTA_API="/home/ChunLi/webapi"
RUTA_EXTERNA="/home/ChunLi/respaldos_locales"
FECHA=$(date +%Y-%m-%d_%H-%M)

# 2. Creamos la carpeta temporal interna para juntar los datos
mkdir -p $RUTA_API/tmp_empaque
cp $RUTA_API/db.json $RUTA_API/tmp_empaque/
cp -r $RUTA_API/img $RUTA_API/tmp_empaque/

# [RESPALDO 1] El archivo dentro de la misma carpeta webapi (Ya funciona)
tar -czf $RUTA_API/respaldo_interno_$FECHA.tar.gz -C $RUTA_API tmp_empaque

# [RESPALDO 2] El archivo NUEVO fuera de la carpeta de la API
tar -czf $RUTA_EXTERNA/respaldo_externo_$FECHA.tar.gz -C $RUTA_API tmp_empaque

# 3. Limpiamos la carpeta temporal
rm -rf $RUTA_API/tmp_empaque
```

Figura 54. Modificación del archivo.

Demostración: Ejecutamos de nueva manera el script para comprobar el cambio, y por medio de un “ls” comprobamos que se haya generado (ver figura 55).

```
ChunLi@vultr:~$ /bin/bash /home/ChunLi/respaldo_321.sh
ChunLi@vultr:~$
ChunLi@vultr:~$ ls -lh /home/ChunLi/respaldos_locales/
total 12K
-rw-rw-r-- 1 ChunLi ChunLi 11K May 24 10:27 respaldo_externo_2026-05-24_10-27.tar.gz
ChunLi@vultr:~$
```

Figura 55. Modificación del archivo.

8. RESPALDO HACIA LA NUBE (GOOGLE DRIVE)

Paso 1. Ingresamos a la página de “Google Developers” a través del siguiente link: <https://developers.google.com/oauthplayground/> (ver figura 56).

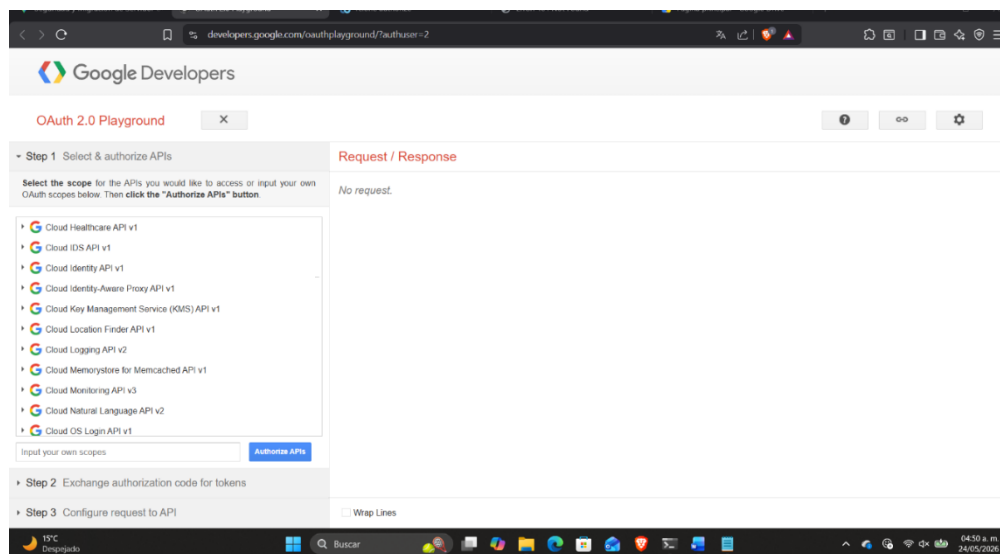


Figura 56. Google developers.

Paso 2. En la lista de la izquierda busca **Drive API v3** y selecciona el cuadrito que dice <https://www.googleapis.com/auth/drive.file> (ver figura 57).

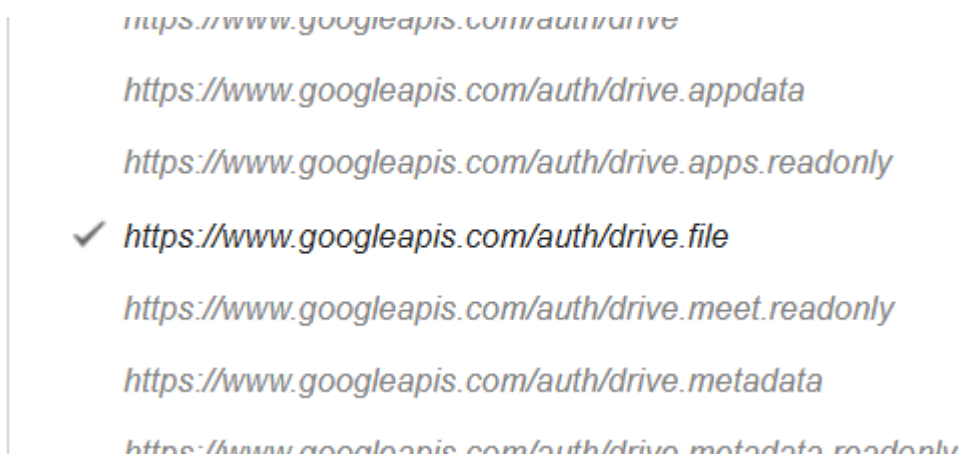


Figura 57. Opción a elegir.

Paso 3. Una vez escogida la opción, le damos click izquierdo al botón "Authorize APIs", elegimos una cuenta de Google que queramos usar y damos click izquierdo a "Permitir" (ver figura 58).

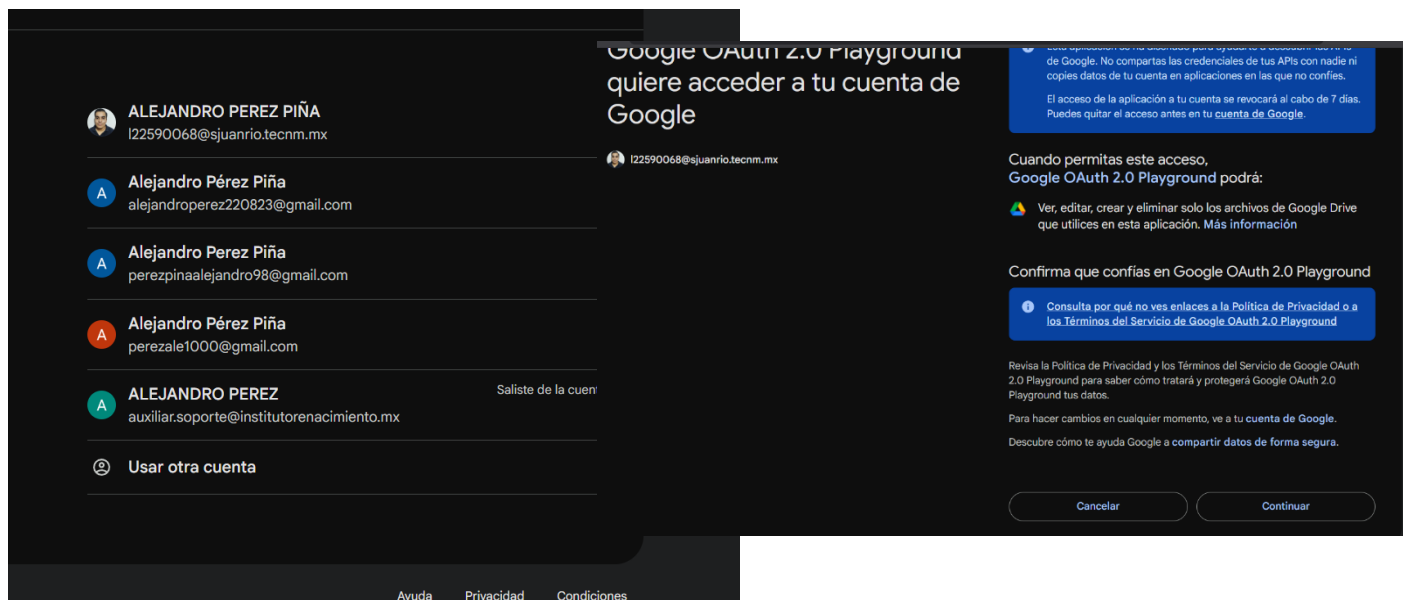


Figura 58. Opciones a elegir.

Paso 4. En el panel izquierdo, damos click izquierdo al botón "Exchange authorization code for tokens" (ver figura 59).

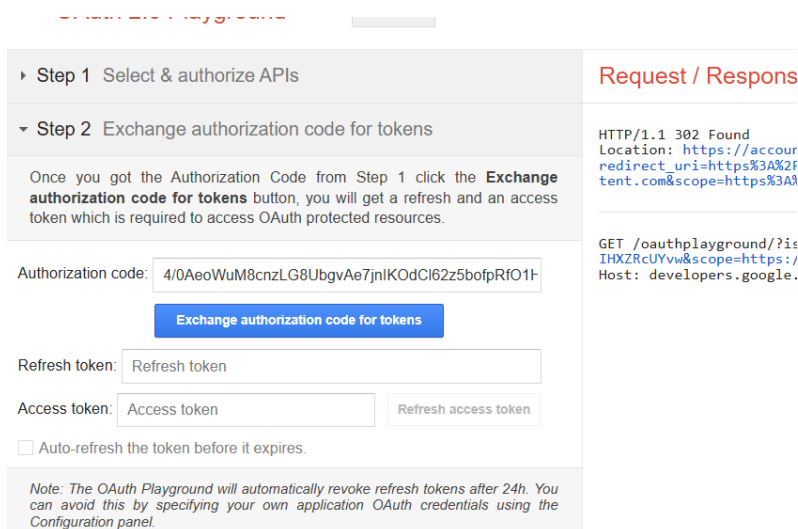


Figura 59. Botón correspondiente.

Paso 5. En el panel derecho, buscamos la parte que diga “access_token” y la guardamos porque se va a utilizar (ver figura 60).

```

der of X-frame-options: SAMEORIGIN
Alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
Content-type: application/json; charset=utf-8

{
  "access_token": "ya29.a0AQvPyIPv2ySHXDrPkv5n_ekgwcxgdrch0xD_-TSHYpuf2xsbmJIYto1PjBl0enTWRGKIo-Jwzq08nmcu30Tz5IfM-pteVZzkvdIoqg0EP_QgNMkdAXBC6ya0hN1b1kIdMz0FEA7fsq5yuQ0Q2JCsRtQkNHDJvQ2_7lps4IBz28yDM3BHX8LIkSm05n8tKT_xRFgaCgYKASQSARESFQ7FMp5A0206",
  "refresh_token_expires_in": 604798,
  "expires_in": 3599,
  "token_type": "Bearer",
  "scope": "https://www.googleapis.com/auth/drive.file"
}

```

Figura 60. access_token.

Paso 6. En el archivo que creamos en la sección 6, paso 2 de este documento, lo modificamos para que realice un POST con nuestro token generado en el paso anterior (ver figura 61). Guardamos presionando las teclas “Ctrl + O”, luego “Enter” para confirmar, y “Ctrl + X”.

```

# 1. Definimos las rutas y la fecha
RUTA_API="/home/ChunLi/webapi"
RUTA_EXTERNA="/home/ChunLi/respaldos_locales"
FECHA=$(date +%Y-%m-%d_%H-%M)

# 2. Creamos la carpeta temporal interna para juntar los datos
mkdir -p $RUTA_API/tmp_empaque
cp $RUTA_API/db.json $RUTA_API/tmp_empaque/
cp -r $RUTA_API/img $RUTA_API/tmp_empaque/

# [RESPALDO 1] El archivo dentro de la misma carpeta webapi
tar -czf $RUTA_API/respaldo_interno_$FECHA.tar.gz -C $RUTA_API tmp_empaque

# [RESPALDO 2] El archivo fuera de la carpeta de la API
tar -czf $RUTA_EXTERNA/respaldo_externo_$FECHA.tar.gz -C $RUTA_API tmp_empaque

# 3. Limpiamos la carpeta temporal
rm -rf $RUTA_API/tmp_empaque

# [RESPALDO 3] SUBIDA DIRECTA A GOOGLE DRIVE SIN RCLONE
# (Usa curl para mandarlo a la cuenta dueña del Token)
curl -X POST \
-H "Authorization: Bearer ya29.a0AQvPyIPv2ySHXDrPkv5n_ekgwcxgdrch0xD_-TSHYpuf2xsbmJIYto1PjBl0enTWRGKIo-Jwzq08nmcu30Tz5IfM-pteTNa-VZzkvdIoqg0EP_QgNMkdAXBC" \
-F "metadata={name: 'respaldo_externo_$FECHA.tar.gz';type=application/json;charset=UTF-8}" \
-F "file=$RUTA_EXTERNA/respaldo_externo_$FECHA.tar.gz" \
"https://www.googleapis.com/upload/drive/v3/files?uploadType=multipart"

```

Figura 61. Archivo corregido.

Demostración: Ejecutamos el script para que envíe el respaldo a Google Drive (ver figura 62).

```

ChunLi@vultr:~$ /bin/bash /home/ChunLi/respaldo_321.sh
{
  "kind": "drive#file",
  "id": "1dKT6ru1QUGphGpTN6jo0Z0gvpDd9_imj",
  "name": "respaldo_externo_2026-05-24_10-57.tar.gz",
  "mimeType": "application/x-gzip"
}

```

Figura 62. Ejecución del archivo.

Y verificamos en Google drive si se guardo el respaldo (ver figura 63).

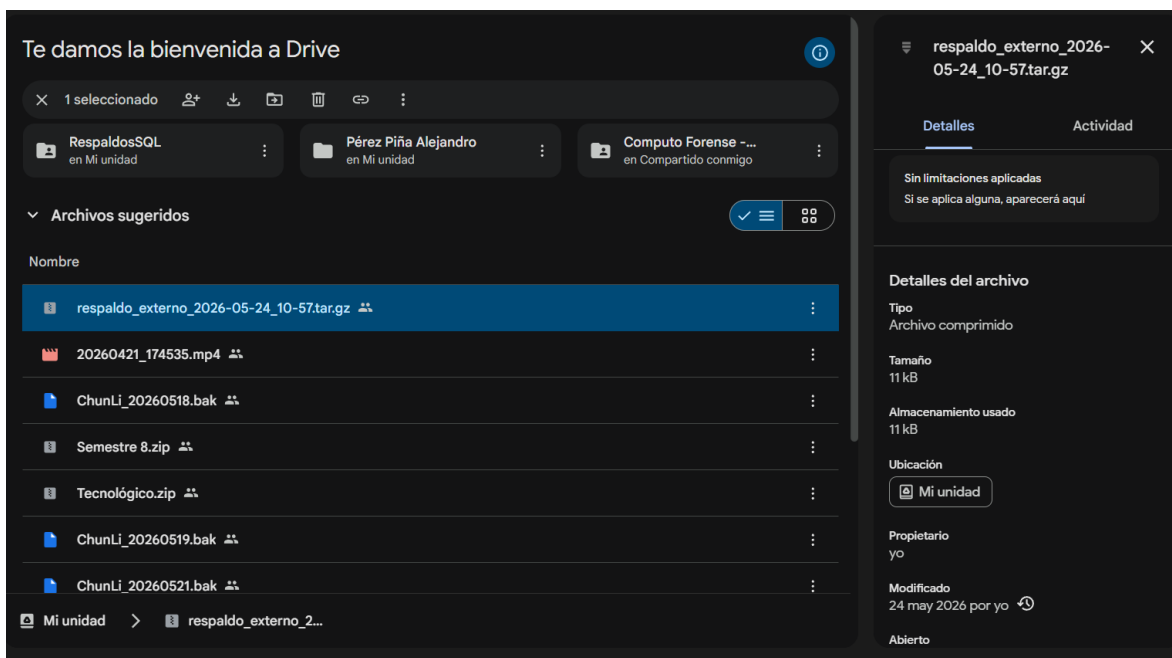


Figura 63. Respaldo en Google Drive.